Capacity Monitoring and Planning

watch the trend, act before the limit

Watch the 4-min video

In one line: track utilisation and saturation as trends against their limits, including cloud quotas you don't control, forecast when each resource will cross a headroom threshold, and add capacity before it does; alert on predictions, which give weeks of warning instead of minutes.

1 The most predictable outage

Black Friday morning. Traffic triples, the database hits its connection limit, checkout falls over. Looking back, the graphs had been creeping up for months; nobody watched the trend. Running out of capacity is one of the most predictable outages there is, if someone looks ahead.

2 Utilisation, saturation and quotas

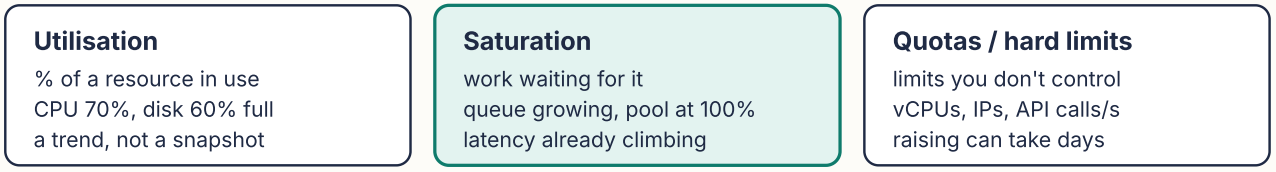


Figure 1. Three things to track for every resource. Saturation is often the first real warning; quotas are the ones most often forgotten.

Utilisation is how much of each resource you use: CPU, memory, disk, network, DB connections, queue depth. Plot it **against its limit** on the same graph, over weeks and months. **Saturation** means work is waiting: a CPU at 70% may be fine, but a connection pool at 100% means requests are already queueing (the USE method, lesson 61). **Quotas**: cloud accounts cap vCPUs per region, public IPs, API calls per second, storage. Hitting one stops scaling instantly, and an increase can take days, so track quota usage like any other resource and request increases early.

3 Sizing from first principles

Little's law: requests in flight = arrival rate × time in system. At 2,000 req/s with 50 ms spent holding a DB connection, you need about 2,000 × 0.05 = **100 connections** busy on average, so a pool of 80 saturates even though CPU looks idle. Measure throughput per instance in a load test, then: instances = peak load ÷ safe per-instance throughput, plus headroom.

4 Forecasting

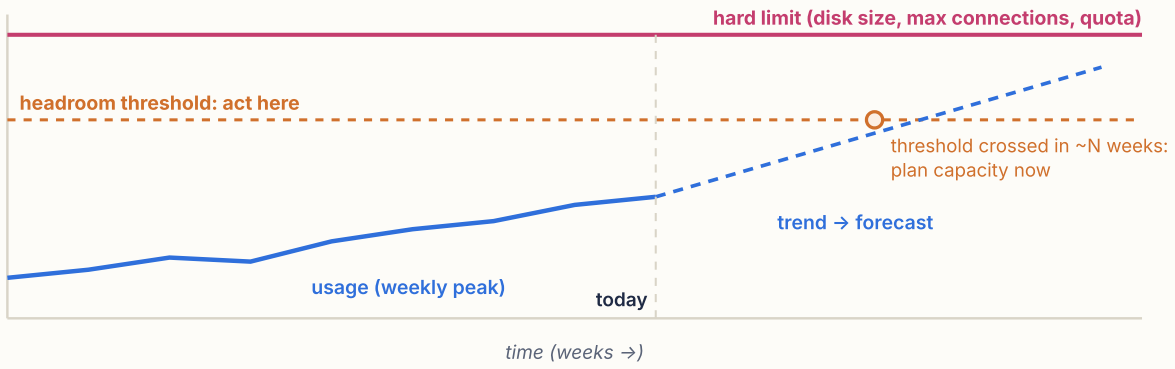


Figure 2. Usage against its limit, with a trend projected forward. The decision point is the headroom threshold, not the hard limit.

Fit a trend to the last few months and project it: if disk grows 2% a week, you know roughly when it fills. Add **known events**: launches, marketing campaigns, seasonal peaks (last year's Black Friday × growth). Forecasts are never exact, but they turn surprises into planned work. Review every quarter and compare last quarter's forecast with reality to calibrate.

5 Headroom policy

Decide how much spare capacity you always hold, e.g. **30–50%**, to absorb spikes and the loss of a zone. With 3 zones, surviving one zone loss means the other two carry 1.5× their load, so each must run below about **66%** at peak. When usage crosses the threshold, add capacity or improve efficiency. Autoscaling covers stateless compute, but databases, quotas and licences still need planning. Too much headroom wastes money: revisit it.

Resource	Leading signal	Lead time to add
Stateless compute	CPU, request rate per pod	minutes (autoscaling)
DB connections	pool wait time, active / max	hours to days (pooler, bigger instance)
Disk / storage	days until full	hours to weeks
Cloud quotas	used / quota	days (support request)

WORKED EXAMPLE: ALERT WHEN THE DISK WILL BE FULL IN 30 DAYS

The orders database has a **2 TB** disk. Instead of alerting at 90% full (minutes of warning on a busy night), the team alerts on a prediction: `predict_linear(node_filesystem_avail_bytes{mountpoint="/data"}[7d], 30*24*3600) < 0`, a linear fit over the last 7 days that asks "will free space hit zero within 30 days?". It opens a **ticket**, not a page. It fires on a calm Tuesday, giving weeks to add storage or archive old orders.

COMMON MISTAKES

- Static 90% thresholds that fire too late on fast growth and nag forever on slow growth.
- Watching CPU while the connection pool or a queue is the real bottleneck.
- Forgetting cloud quotas until a launch-day scale-out fails.
- Planning for average load instead of peak plus a zone failure.

6 Key terms

Utilisation: share of a resource in use

Quota: provider-imposed limit, e.g. vCPUs per region

Forecast: projected usage from trend plus known events

Saturation: work waiting because a resource is full

Headroom: spare capacity held on purpose

predict_linear: PromQL linear extrapolation of a series

7 Interview questions

Q1. How do you know when to add capacity? BASIC

Track utilisation and saturation trends against limits, watch quotas, forecast when each resource crosses its headroom threshold (including launches and peaks), and add capacity before that point.

Q2. Why alert on predictions instead of fixed thresholds? BASIC

They give weeks of warning instead of minutes, and adapt to the growth rate: fast growth alerts early, slow growth doesn't nag.

Q3. CPU is at 40% but latency doubles at peak. What do you check? SCENARIO

Saturation elsewhere: connection-pool wait, thread pools, queue depth, a downstream dependency or lock contention. Low CPU doesn't mean spare capacity.

Q4. A launch will bring 5× traffic in a month. How do you prepare? SCENARIO

Load-test to find per-instance limits and the first bottleneck, check quotas and request increases now, pre-scale stateful tiers, and keep headroom for a zone loss.

Q5. Doesn't autoscaling make capacity planning unnecessary? TRICKY

No: it scales stateless compute within quotas, but databases, connection limits, quotas and cold-start time still need planning ahead.

Q6. Why keep each of three zones below ~66%? TRICKY

If one zone fails, the remaining two take its load: 1.5× each. Above about 66% that pushes them past 100%.

Next: SD-N-014 · CI/CD Pipelines backendroadmap.com